Prioridad del día

Lab 12

Problema 1:

a) Reducción - B de la operación NoT

Regla de la reducción - B:

$$(\lambda x.M)\ N \rightarrow_\beta M[x := N]$$

Booleanos de Church: ξ

$$TRUE \doteq \lambda x.\lambda y.x$$

$$False = \lambda x.\lambda y.y$$

NoT tiene que devolver False si p es True y True si p es False, entonces se usa a p para elegir

$$NoT = \lambda p.\ p\ False\ True$$

No tienes que ser rico para tener éxito, pero tienes que tener éxito para ser rico.

Prioridad del día

Caso 1: Not True

NOT True = $(\lambda p.p\ False\ True)(\lambda x.\lambda y.$

$\rightarrow_\beta (\lambda x \lambda y.x)\ False\ True \quad [p := True]$

$\rightarrow_\beta (\lambda y.False)\ True \quad [x := False]$

$\rightarrow_\beta False \quad [y := True$ y no aparece$]$

$= \lambda x.\lambda y.y \quad \therefore$ Not True = False

Caso 2: Not False

Not False $= (\lambda p.p\ False\ True)(\lambda x.\lambda y.y)$

$\rightarrow_\beta (\lambda x.\lambda y.y)\ False\ True \quad [p := False]$

$\rightarrow_\beta (\lambda y.y)\ True \quad [x := False$, x no aparece$]$

$\rightarrow_\beta True \quad [y := True]$

$= \lambda x.\lambda y.x \quad \therefore$ Not False = True

NOTAS

El éxito es la capacidad de ir de fracaso en fracaso sin perder el entusiasmo.

Winston Churchill

Prioridad del día

Caso 3: NOT True con todo expandido

$(\lambda p.p\ (\lambda a.\lambda b.b)\ (\lambda a.\lambda b.a))\ (\lambda x.\lambda y.x)$

$\rightarrow_\beta (\lambda x.\lambda y.x)\ (\lambda a.\lambda b.b)\ (\lambda a.\lambda b.a)$ $[p := True]$

$\rightarrow_\beta (\lambda y.\lambda a.\lambda b.b)\ (\lambda a.\lambda b.a)$ $[x := \lambda a.\lambda b.b]$

$\rightarrow_\beta \lambda a.\lambda b.b$ $[y := \lambda a.\lambda b.a$ no aparece$]$

$\equiv \lambda x.\lambda y.y = False$

Otra forma NOT que intercambia las opciones

$NOT = \lambda p.\lambda a.\lambda b.p\ b\ a$

$NOT\ True = (\lambda p.\lambda a.\lambda b.p\ b\ a)\ (\lambda x.\lambda y.x)$

$\rightarrow_\beta \lambda a.\lambda b.(\lambda x.\lambda y.x)\ b\ a$ $[p := True]$

$\rightarrow_\beta \lambda a.\lambda b.(\lambda y.b)\ a$ $[x := b]$

$\rightarrow_\beta \lambda a.\lambda b.b \equiv False$ $[y := a]$

El éxito es una mentalidad. Si crees que puedes, lo harás.

Marva Collins

MARZO

V	S	D	L	M	M	J	V	S	D	L	M	M	J	V	S	D	L	M	M	J	V	S	D	L	M	M	J	V	S	D
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31

30.31
SÁB . DOM

MOTIVACIÓN

Prioridad del día

Comprobación

p = True → Not p = False

p = False → Not p = True

Not (Not True) = Not False = True

b) Recursión y ciclos:

En el cálculo λ (y la programación funcional pura):

- las funciones son anónimas: no tienen nombre para llamarse.
- No hay variables que cambien ni for / while.

Por eso, la recursión se logra con un combinador de punto fijo y los ciclos, se escriben como recursión.

NOTAS

Cuando pierdas, no pierdas la lección.

Dalai Lama

Prioridad del día

07:

Combinador Y:

08: $Y = \lambda f.(\lambda x.f(x\,x))(\lambda x.f(x\,x))$

09: 1° propiedad: $Y_g = g(Y_g)$ (Y_g es punto fijo de g)

10: Paso 1: $Y_g \to_\beta (\lambda x.g(x\,x))(\lambda x.g(x\,x))\ [f := g]$

11: Paso 2: $\to_\beta g((\lambda x.g(x\,x))(\lambda x.g(x\,x)))$

12: $[x := \lambda x.g(x\,x)]$

01: Paso 3: lo de adentro de g es el

02: resultado del Paso 1, osea Y_g

$\therefore Y_g = g(Y_g)$

03:

Así g se ~~~~ recibe a sí misma

04: cómo parámetro y se puede volver

a llamar: $g(g(g(\ldots)))$

05:

06:

07:

No busques los errores, busca un remedio.

Henry Ford

2

ABRIL
2024

MARTES

MOTIVACIÓN

Prioridad del día

Recursión: factorial

$G = \lambda f.\lambda n.\ If\ (Iszero\ n)\ 1\ (Mult\ n\ (f\ (Pred\ n)))$

$Fact = Y\,G$

- f: la llamada recursiva
- Iszero n: caso base
- Pred n: n − 1

Corrida de Fact 3:

$Fact\ 3 = G\ (Y\,G)\ 3$

$\rightarrow_\beta If\ (Iszero\ 3)\ 1\ (Mult\ 3\ (Fact\ 2))$

$\rightarrow Mult\ 3\ (Fact\ 2)$

$\rightarrow Mult\ 3\ (Mult\ 2\ (Fact\ 1))$

$\rightarrow Mult\ 3\ (Mult\ 2\ (Mult\ 1\ (Fact\ 0)))$

$\rightarrow Mult\ 3\ (Mult\ 2\ (Mult\ 1\ 1))$ (caso base)

$\rightarrow 6$

La vida es una aventura, atrévete.

Teresa de Calcuta

Prioridad del día

Ciclos: Un ciclo while

Mientras cond(s): s = paso(s)

Se vuelve recursión de cola: el estado s se pasa como parámetro, en vez de modificarlo.

Loop = Y(λl.λs. If (cond s) (l (paso s)) s)

Ejemplo: suma 1+2+...+n con acumulador

Suma = Y(λf.λn. λac. If (Iszero n) ac
(f (Pred n) (Add ac n)))

Corrida Sima 3 0:

Suma 3 0 → Suma 2 3 → Suma 1 5

→ Suma 0 6 → 6

Cada vuelta del ciclo es una llamada

n y ac son las variables del ciclo y nunca se modifican

Tu actitud, no tu aptitud, determinará tu altitud.

Zig Ziglar

4

2024

JUEVES

MOTIVACIÓN

Prioridad del día

Ciclo repetir n veces con numerales de Church:

$n = \lambda f.\lambda x.\ f(f(\ldots f(x)))$ (f aplicada n veces)

$3 = \lambda f.\lambda x.\ f(f(f(x)))$

3 succ 0 → succ (succ (succ 0))) = 3

2. el número ya es ciclo for de n vueltas.

En un lenguaje funcional:

factorial 0 = 1

factorial n = n * factorial (n-1)

suma n = foldl' (+) 0 [1..n]

map, filter y foldl reemplazaron a los for de siempre.

Tienes que hacer las cosas que crees que no puedes hacer.

Eleanor Roosevelt

NOTAS

Prioridad del día

c) ¿Cuándo usar programación funcional y cuándo no?

Es prudente cuando:

- Se transforman datos (listas, colecciones) con map, filter, fold
- Hay concurrencia o paralelismo: no hay estado mutable compartido → no hay condiciones de carrera.
- Se requiere código fácil de probar: misma entrada → misma salida
- Compiladores, parser y cálculos matemáticos

Ejemplo: total de ventas de noviembre

total = ventas.map monto . filter (_.mes == 11)

Cada paso es una función pura: se pueden probar por separado y paralelizar.

Los problemas 2, 3 y 4 son de este tipo.

Si te caíste ayer, levántate hoy.

H. G. Wells

6 . 7

SÁB . DOM

ABRIL 2024

MOTIVACIÓN

No es prudente cuando: Prioridad del día

- el programa es casi todo estado de cambio e I/O
- hay poca memoria o tiempo real (embebidos): la inmutabilidad crea copias, hay recolector de basura y la recursión gasta pila
- se modifican en el lugar arreglos o matrices enormes
- el equipo no conoce el paradigma.

Ejemplo: firmware de microcontrolador (Arduino) que lee un sensor y prende un ventilador.

loop: leer T → si T > 30 prender pin → repetir

Tiene muy poca RAM, debe responder en tiempo fijo y todo su trabajo es cambiar registros y pines (estado)
→ conviene C imperativo.

NOTAS

Siempre parece imposible... hasta que se hace.

Nelson Mandela